Baseline Reproduction — Technical Memo

Group 3: *Kharintirasakar Uthayanan, David Zajac, Diana Riyahi, Shawn Xiao, Gamze Esen-Erdemir*

Date: *October 19, 2026*

Subject: *Baseline Reproduction — Classical Baselines on CantoNLU (Sentiment Analysis and Language Detection)*

1. Repository Submission

- ***Repository URL:*** *CantoNLU*
- ***Release tag:*** *v0.3-m3-baseline*

2. Reproducible Environment & Repository Setup

The code for this milestone is tagged as v0.3-m3-baseline. It was developed and tested with Python 3.13.14; all package versions are pinned in requirements.txt (e.g. scikit-learn 1.7.2, pandas 2.3.3, numpy 2.2.6) and installed with pip install -r requirements.txt. The CantoNLU data (CC BY 4.0) is not redistributed; data/README.md gives the exact commands to copy it from the upstream repository. All randomness is fixed with seed 3052 (seeds 3052–3054 for the stratified benchmark), and the full baseline table is regenerated with one command, python src/train_baselines.py, which writes results/baseline_valid.csv. Three team members re-ran it on their own machines (Windows; Python 3.10, 3.11 and 3.12); all values matched the committed table except one value pair, discussed in Section 6.

3. Working Data Pipeline

The pipeline is split into two modules in src/:

- ***Ingestion and checks (src/data.py):*** *loads the official CantoNLU train/validation/test splits for both tasks, unchanged. We do not re-split: the splits are the ones used by the published models, and the language-detection splits are grouped by source sentence, which we verified in Milestone 2. Every load checks that the expected columns exist, that no sentence is missing or empty, and that only the expected labels occur, so a wrong or corrupted file fails immediately. Paths are resolved relative to the repository, with no personal paths.*
- ***Preprocessing and training (src/train_baselines.py):*** *each sentence is represented by character 1–3-gram TF-IDF features, which avoids a separate Cantonese word-segmentation step. The vectoriser and the classifier form one sklearn Pipeline, so the vectoriser is fitted on the training split only. All models go through the same fit-on-train, evaluate-on-validation function.*
- ***Locked test split:*** *the test split is only read with the --final flag, which prints a warning and is reserved for the single final evaluation. Unlike the template, the table below therefore reports validation results.*

4. Baseline Methodology

- ***Naive benchmarks:*** *majority class (DummyClassifier(strategy="most_frequent")) and stratified random guessing (strategy="stratified", three seeds). Neither model looks at the text.*
- ***Course-level baselines:*** *Multinomial Naive Bayes and L2-regularised logistic regression (LogisticRegression(max_iter=1000)) on the TF-IDF features, with default hyperparameters (NB alpha = 1.0, LR C = 1.0). Tuning on the validation split is the next step.*

5. Baseline Table

Validation split. Macro-F1 is the primary metric (imbalanced language-detection classes); micro- and weighted-F1 are added because the CantoNLU paper does not state which F1 average it reports. Source: results/baseline_valid.csv.

Task	*Model*	*Accuracy*	*Macro-F1*	*Micro-F1*	*Weighted-F1*
Sentiment	*Majority class*	*0.333*	*0.167*	*0.333*	*0.167*
Sentiment	*Stratified random (3 seeds)*	*0.323–0.334*	*0.323–0.334*	*0.323–0.334*	*0.323–0.334*
Sentiment	*TF-IDF + Naive Bayes*	*0.698*	*0.686*	*0.698*	*0.686*
Sentiment	*TF-IDF + Logistic Regression*	*0.751*	*0.751*	*0.751*	*0.751*
Lang. detection	*Majority class*	*0.671*	*0.268*	*0.671*	*0.539*
Lang. detection	*Stratified random (3 seeds)*	*0.503–0.523*	*0.320–0.345*	*0.503–0.523*	*0.499–0.518*
Lang. detection	*TF-IDF + Naive Bayes*	*0.671*	*0.268*	*0.671*	*0.539*

Lang. detection	*TF-IDF + Logistic Regression*	*0.756*	*0.599*	*0.756*	*0.721*

6. Discrepancy Analysis

- ***Expected vs. reproduced (naive benchmarks):*** *the majority classifier on language detection gives accuracy 0.671 and macro-F1 0.268, exactly the values expected from the class distribution (67.1% code-mixed; F1 of 0.80 for one class and 0 for the other two). Stratified guessing gives macro-F1 close to 1/3 on both tasks, as expected for random guessing in proportion to the classes. This confirms that the pipeline and metrics behave as intended.*
- ***Reproduction across machines:*** *on the other machines, language-detection logistic regression gave macro-/weighted-F1 of 0.600 / 0.722 instead of the committed 0.599 / 0.721; all other values matched. The difference persisted when the committed table was re-generated in a fresh environment with the pinned package versions, so it comes from floating-point differences in the optimiser across environments (Python build, CPU-specific BLAS kernels). The unrounded values (0.59915, 0.72148) lie just below the rounding boundaries, so one changed prediction can change the third decimal. We now also pin scipy, which provides the solver, and report the variation instead of hiding it.*
- ***Comparison with published results:*** *the CantoNLU paper reports F1 of 70.1–71.9 (sentiment) and 73.3–78.7 (language detection) for four pretrained models on the test split. On sentiment, our logistic regression reaches 0.751 on validation with all three F1 averages (with balanced classes, macro- and weighted-F1 are always equal), so the F1 definition hardly matters here. This does not show that the classical model beats the pretrained models: our scores are on validation, theirs on test. On language detection the conclusion depends on the F1 average: micro-F1 (0.756) lies inside the published range, while macro-F1 (0.599) is far below it. In addition, the released language-detection data has more code-mixed sentences than the paper reports (32,033 vs. 27,578), so the class distribution may differ from the one used for the published scores.*
- ***Naive Bayes fails on language detection:*** *it predicts the code-mixed class for all 2,425 validation sentences, so it does not beat the majority benchmark. Removing the class prior (fit_prior=False) changes only 11 predictions (scripts/nb_ld_check.py), so the 67% prior is not the main cause. Our hypothesis is that the classes differ by only a few characters, code-mixed sentences contain the characters of both languages, and smoothing (alpha = 1) over a very large n-gram vocabulary affects the smaller Cantonese and Mandarin classes more. We will test this by tuning alpha on the validation split.*

7. Start / Stop / Continue Plan

- ***Start:*** *hyperparameter search on the validation split (NB alpha, LR C, n-gram range) with the search space logged; kNN on frozen LaBSE embeddings as the embedding-based model; reporting mean ± standard deviation over seeds from the script itself; error analysis of the logistic-regression confusion matrix.*
- ***Stop:*** *leaving one-off scripts in the repository root (moved to scripts/), and documenting commands in the README before they exist (--final was documented before it was implemented).*
- ***Continue:*** *one branch and one reviewed pull request per task; keeping the test split locked; regenerating every reported number from a script and checking it against the committed results table.*

AI assistance used for this milestone is logged in AI_USE.md.